```
pc4278_nyu_edu@nyu-dataproc-m:~/6_HW$ spark-shell --master yarn --deploy-mode client < Clean.s
cala
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/04/05 05:52:57 INFO SparkEnv: Registering MapOutputTracker
26/04/05 05:52:57 INFO SparkEnv: Registering BlockManagerMaster
26/04/05 05:52:57 INFO SparkEnv: Registering BlockManagerMasterHeartbeat
26/04/05 05:52:57 INFO SparkEnv: Registering OutputCommitCoordinator
Spark context Web UI available at http://nyu-dataproc-m.c.hpc-dataproc-19b8.internal:44255
Spark context available as 'sc' (master = yarn, app id = application_1768446951072_8972).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.5.3
      /_/

Using Scala version 2.12.18 (OpenJDK 64-Bit Server VM, Java 11.0.20.1)
Type in expressions to have them evaluated.
Type :help for more information.

scala> import org.apache.spark.sql.functions._
import org.apache.spark.sql.functions._

scala> import org.apache.spark.sql.types._
import org.apache.spark.sql.types._

scala> val hdfsBase = "6_HW/ingested"
hdfsBase: String = 6_HW/ingested

scala> val outBase  = "6_HW/cleaned"
outBase: String = 6_HW/cleaned

scala>
```

```
scala> // 1. CORRIDORS

scala> println("Cleaning corridors...")
Cleaning corridors...

scala> val corridorsRaw = spark.read.option("header","true").option("inferSchema","true").csv(
s"${hdfsBase}/corridors")
corridorsRaw: org.apache.spark.sql.DataFrame = [year: int, origin_region: string ... 3 more fi
elds]

scala> println(s"  Raw: ${corridorsRaw.count()}")
  Raw: 4863

scala> val corridorsClean = corridorsRaw.filter(col("value").isNotNull).filter(col("origin_reg
ion").isNotNull && col("destination_region").isNotNull).filter(col("origin_region")=!=col("des
tination_region")).filter(col("year").between(2019,2024))
corridorsClean: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [year: int, origin_re
gion: string ... 3 more fields]

scala> println(s"  Cleaned: ${corridorsClean.count()}")
  Cleaned: 4863

scala> corridorsClean.write.mode("overwrite").option("header","true").csv(s"${outBase}/corrido
rs")
```

```
scala> // 2. REGIONAL

scala> println("Cleaning regional...")
Cleaning regional...

scala> val regionalRaw = spark.read.option("header","true").option("inferSchema","true").csv(s
"${hdfsBase}/regional")
regionalRaw: org.apache.spark.sql.DataFrame = [year: int, region: string ... 3 more fields]

scala> println(s"  Raw: ${regionalRaw.count()}")
  Raw: 2844

scala> val regionalClean = regionalRaw.filter(col("value").isNotNull).filter(col("region").isN
otNull).filter(col("year").between(2019,2024))
regionalClean: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [year: int, region: st
ring ... 3 more fields]

scala> println(s"  Cleaned: ${regionalClean.count()}")
  Cleaned: 2844

scala> regionalClean.write.mode("overwrite").option("header","true").csv(s"${outBase}/regional
")
```

```
scala> // 3. CARGO

scala> println("Cleaning cargo...")
Cleaning cargo...

scala> val cargoRaw = spark.read.option("header","true").option("inferSchema","true").csv(s"${
hdfsBase}/cargo")
cargoRaw: org.apache.spark.sql.DataFrame = [year: int, region: string ... 4 more fields]

scala> println(s"  Raw: ${cargoRaw.count()}")
  Raw: 3478

scala> val cargoClean = cargoRaw.filter(col("value").isNotNull && col("value")>0).filter(col("
region").isNotNull).filter(col("cargo_class").isNotNull).filter(col("year").between(2019,2024)
)
cargoClean: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [year: int, region: strin
g ... 4 more fields]

scala> println(s"  Cleaned: ${cargoClean.count()}")
  Cleaned: 3478

scala> cargoClean.write.mode("overwrite").option("header","true").csv(s"${outBase}/cargo")
```

```
scala> // 4. PRICE INDEX

scala> println("Cleaning price_index...")
Cleaning price_index...

scala> val priceRaw = spark.read.option("header","true").option("inferSchema","true").csv(s"${
hdfsBase}/price_index")
priceRaw: org.apache.spark.sql.DataFrame = [year: int, quarter: string ... 2 more fields]

scala> println(s"  Raw: ${priceRaw.count()}")
  Raw: 144

scala> val priceClean = priceRaw.filter(col("price_index").isNotNull).filter(col("year").betwe
en(2019,2024))
priceClean: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [year: int, quarter: stri
ng ... 2 more fields]

scala> println(s"  Cleaned: ${priceClean.count()}")
  Cleaned: 144

scala> priceClean.write.mode("overwrite").option("header","true").csv(s"${outBase}/price_index
")
```

```
scala> // 5. DISTANCE BANDS

scala> println("Cleaning distance_bands...")
Cleaning distance_bands...

scala> val distRaw = spark.read.option("header","true").option("inferSchema","true").csv(s"${h
dfsBase}/distance_bands")
distRaw: org.apache.spark.sql.DataFrame = [year: int, distance_band: string ... 3 more fields]

scala> println(s"  Raw: ${distRaw.count()}")
  Raw: 420

scala> val distClean = distRaw.filter(col("value").isNotNull && col("value")>0).filter(col("di
stance_band").isNotNull)
distClean: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [year: int, distance_band:
 string ... 3 more fields]

scala> println(s"  Cleaned: ${distClean.count()}")
  Cleaned: 420

scala> distClean.write.mode("overwrite").option("header","true").csv(s"${outBase}/distance_ban
ds")
```

```
scala> // 6. ACOTRAM

scala> println("Cleaning acotram...")
Cleaning acotram...

scala> val acotramRaw = spark.read.option("header","true").option("inferSchema","true").csv(s"
${hdfsBase}/acotram")
acotramRaw: org.apache.spark.sql.DataFrame = [period: date, vehicle_type: string ... 17 more f
ields]

scala> println(s"  Raw: ${acotramRaw.count()}")
  Raw: 31

scala> val acotramClean = acotramRaw.drop("vehicle_type","vehicle_price_eur","semitrailer_pric
e_eur","interest_rate_pct").withColumn("year",substring(col("period").cast("string"),1,4).cast
(IntegerType)).filter(col("year").between(2019,2024)).drop("year").filter(col("period").isNotN
ull)
acotramClean: org.apache.spark.sql.Dataset[org.apache.spark.sql.Row] = [period: date, km_anual
: double ... 13 more fields]

scala> println(s"  Cleaned: ${acotramClean.count()}")
  Cleaned: 23

scala> acotramClean.write.mode("overwrite").option("header","true").csv(s"${outBase}/acotram")
```

```
scala> // SUMMARY

scala> println("== CLEANING COMPLETE ==")
== CLEANING COMPLETE ==

scala> Seq("corridors","regional","cargo","price_index","distance_bands","acotram").foreach{ds
=>val df=spark.read.option("header","true").option("inferSchema","true").csv(s"${outBase}/${ds
}");println(s"  $ds: ${df.count()} records, ${df.columns.length} columns")}
  corridors: 4863 records, 5 columns
  regional: 2844 records, 5 columns
  cargo: 3478 records, 6 columns
  price_index: 144 records, 4 columns
  distance_bands: 420 records, 5 columns
  acotram: 23 records, 15 columns

scala>

scala> // HIVE TABLES

scala> println("Creating Hive tables...")
Creating Hive tables...

scala> spark.sql("CREATE DATABASE IF NOT EXISTS flota LOCATION 'hdfs:///user/bc4278_nyu_edu/fl
ota.db'")
26/04/05 05:53:46 WARN HiveConf: HiveConf of name hive.vectorized.use.checked.expressions does
 not exist
26/04/05 05:53:46 WARN HiveConf: HiveConf of name hive.vectorized.input.format.excludes does n
ot exist
ivysettings.xml file not found in HIVE_HOME or HIVE_CONF_DIR,/etc/hive/conf.dist/ivysettings.x
ml will be used
res27: org.apache.spark.sql.DataFrame = []

scala> Seq("corridors","regional","cargo","price_index","distance_bands","acotram").foreach {
ds =>
     |   spark.read.option("header","true").option("inferSchema","true").csv(s"${outBase}/${ds
}").write.mode("overwrite").saveAsTable(s"flota.${ds}")
     | }
26/04/05 05:53:56 WARN SessionState: METASTORE_FILTER_HOOK will be ignored, since hive.securit
y.authorization.manager is set to instance of HiveAuthorizerFactory.
```

```
Hive tables created:

scala> spark.sql("SHOW TABLES IN flota").show()
+---------+--------------+-----------+
|namespace|     tableName|isTemporary|
+---------+--------------+-----------+
|    flota|       acotram|      false|
|    flota|         cargo|      false|
|    flota|     corridors|      false|
|    flota|distance_bands|      false|
|    flota|   price_index|      false|
|    flota|      regional|      false|
+---------+--------------+-----------+


scala> Seq("corridors","regional","cargo","price_index","distance_bands","acotram").foreach {
ds =>
     |   println(s"== DESCRIBE flota.${ds} ==")
     |   spark.sql(s"DESCRIBE flota.${ds}").show()
     | }
== DESCRIBE flota.corridors ==
+------------------+---------+-------+
|          col_name|data_type|comment|
+------------------+---------+-------+
|              year|      int|   NULL|
|     origin_region|   string|   NULL|
|destination_region|   string|   NULL|
|            metric|   string|   NULL|
|             value|   double|   NULL|
+------------------+---------+-------+
```

```
== DESCRIBE flota.regional ==
+-----------------+---------+-------+
|         col_name|data_type|comment|
+-----------------+---------+-------+
|             year|      int|   NULL|
|           region|   string|   NULL|
|displacement_type|   string|   NULL|
|           metric|   string|   NULL|
|            value|   double|   NULL|
+-----------------+---------+-------+

== DESCRIBE flota.cargo ==
+-----------+---------+-------+
|   col_name|data_type|comment|
+-----------+---------+-------+
|       year|      int|   NULL|
|     region|   string|   NULL|
|  direction|   string|   NULL|
|cargo_class|   string|   NULL|
|     metric|   string|   NULL|
|      value|   double|   NULL|
+-----------+---------+-------+

== DESCRIBE flota.price_index ==
+-------------+---------+-------+
|     col_name|data_type|comment|
+-------------+---------+-------+
|         year|      int|   NULL|
|      quarter|   string|   NULL|
|distance_band|   string|   NULL|
|  price_index|   double|   NULL|
+-------------+---------+-------+
```

```
== DESCRIBE flota.distance_bands ==
+-------------+---------+-------+
|     col_name|data_type|comment|
+-------------+---------+-------+
|         year|      int|   NULL|
|distance_band|   string|   NULL|
| service_type|   string|   NULL|
|       metric|   string|   NULL|
|        value|   double|   NULL|
+-------------+---------+-------+

== DESCRIBE flota.acotram ==
+--------------------+---------+-------+
|            col_name|data_type|comment|
+--------------------+---------+-------+
|              period|     date|   NULL|
|            km_anual|   double|   NULL|
|           pct_carga|   double|   NULL|
|         km_en_carga|   double|   NULL|
|       horas_anuales|   double|   NULL|
|personal_conducci...|   double|   NULL|
|dietas_conductor_eur|   double|   NULL|
|         seguros_eur|   double|   NULL|
| costes_fiscales_eur|   double|   NULL|
|          peajes_eur|   double|   NULL|
|            urea_eur|   double|   NULL|
|costes_indirectos...|   double|   NULL|
|fuel_price_per_litre|   double|   NULL|
|fuel_consumption_...|   double|   NULL|
|     combustible_eur|   double|   NULL|
+--------------------+---------+-------+
```